

40-Day DevOps/SRE Coding Roadmap

Tailored for Google, Apple, and Goldman Sachs

Built from your earlier 40-day SRE/DevOps coding roadmap, then reshaped company-by-company so the prep matches how these interview styles usually feel in practice.

Google	Apple	Goldman Sachs
Harder algorithmic pressure, more insistence on clean communication, deeper reasoning and test coverage.	Pragmatic coding, debugging, implementation detail, systems and reliability instincts matter more.	Fast online assessment rhythm, fundamentals, clean coding under time pressure, fewer theatrics.

What changed from the generic version: this plan keeps the strong pattern core from your uploaded roadmap — graphs, heaps, windows, intervals, trees, tries, binary search, cache/rate-limiter design — but changes the pacing, mock style, and emphasis by company.

Google lens: Prioritize problem solving quality, articulation, tradeoffs, correctness, edge cases, and structured coding over raw speed.

Apple lens: Prioritize implementation quality, systems realism, debugging mentality, and being able to turn a vague problem into robust production-flavored code.

Goldman lens: Prioritize getting to a correct solution quickly, writing clean code in Python, and surviving assessment pressure without freezing.

How to use this plan

- Daily target: 90 to 120 minutes on weekdays, 2 to 3 hours on mock days.
- Every solve is spoken out loud: restate the problem, give brute force, optimize, code, test, state complexity.
- Use Python for all coding rounds unless the company or role explicitly says otherwise.
- Keep one running notebook: problem, pattern, bug, edge case missed, and one production analogy.
- Do not farm random LeetCode. Stay inside the pattern list and repeat weak problems until they are clean.

Interview protocol: 0 to 4 min clarify and frame. 5 to 20 min code with narration. Final 3 min test and state time/space complexity.

Company-specific prep philosophy

Google

Assume stronger algorithmic signal, especially graphs, binary search, trees, intervals, and clean decomposition. Practice asking clarifying questions and narrating invariant changes. Google Careers also points candidates to prep resources and interview materials rather than role-specific trick lists, so your plan should favor fundamentals over memorized company lore.

Apple

Use the job descriptions themselves as signal: current Apple software-engineering listings explicitly mention a common process focused on core fundamentals and programming ability, while Apple also has live DevOps/SRE roles and traffic/network proxying roles that skew practical and systems-adjacent. So prep for coding, but make your code look production-friendly: validation, failure cases, clear function boundaries, and lightweight systems discussion.

Goldman Sachs

Goldman explicitly says Engineering candidates may face a HackerRank assessment and then a video interview, followed by several final-round interviews. That means speed and correctness matter early. You need repeatable templates, clean Python, and comfort solving under timed OA conditions.

40-day calendar

Day	Theme	Main work	Company emphasis
1	Graph BFS/DFS refresh	Solve Number of Islands and Course Schedule. Google priority: invariants. Apple: clean graph representation. GS: 30-min cap each.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
2	Topological sort + cycle detection	Re-solve Course Schedule from memory; explain indegree vs DFS color method.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
3	Weighted graph / heap	Network Delay Time. Focus on why Dijkstra works and when it fails.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
4	Union-Find	Redundant Connection. Explain infra analogy: loops / duplicate linkage.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
5	Matrix BFS	01 Matrix or Walls and Gates. Practice boundary handling without bugs.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.

6	Mock 1	1 graph + 1 graph/heap problem. 50 minutes total. Review narration quality.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
7	Recovery day	Write notes: where you hesitated, which template was fuzzy, what bug repeated.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
8	Heap basics	Kth Largest and K Closest Points. Build a strong heap template.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
9	Merge K sorted lists	Use heap cleanly. Apple lens: readable helper structure.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
10	Task Scheduler	Explain greedy intuition without sounding hand-wavey.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
11	Running median	Two heaps. Google lens: articulate balancing invariants.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
12	Top K frequent	Bucket vs heap tradeoff. Goldman lens: pick fastest correct route.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
13	Mock 2	2 heap-ish problems in 55 minutes, one medium and one medium/hard.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
14	Review + implementation cleanup	Rewrite your two messiest heap solves in cleaner Python.	Google: explain. Apple: implement cleanly. Goldman: stay on timer.
15	Sliding window fixed + variable	Max Average Subarray and Longest Substring Without Repeat.	Google: correctness proof. Apple: realistic rate-limiter framing. Goldman: template speed.
16	Minimum Window Substring	Hard day. Focus on exact validity condition.	Google: correctness proof. Apple: realistic rate-limiter framing. Goldman: template speed.
17	Sliding Window Maximum	Deque pattern. State why heap is worse here.	Google: correctness proof. Apple: realistic rate-limiter framing. Goldman: template speed.
18	Rate limiter design	Code a sliding-window rate	Google: correctness proof.

		limiter. Add infra framing.	Apple: realistic rate-limiter framing. Goldman: template speed.
19	Anagram / window variants	Fast recognition drill: what invalidates the window?	Google: correctness proof. Apple: realistic rate-limiter framing. Goldman: template speed.
20	Mock 3	1 window problem + 1 rate-limiter design follow-up.	Google: correctness proof. Apple: realistic rate-limiter framing. Goldman: template speed.
21	Two pointers	Two Sum II and 3Sum. Practice explaining $O(n^2)$ without apologizing.	Google: justify pattern choice. Apple: code hygiene. Goldman: avoid overthinking.
22	Intervals	Merge Intervals and Insert Interval. These are very Goldman-friendly and broadly useful.	Google: justify pattern choice. Apple: code hygiene. Goldman: avoid overthinking.
23	Fast/slow pointer	Linked List Cycle. Tiny problem, but clean explanation matters.	Google: justify pattern choice. Apple: code hygiene. Goldman: avoid overthinking.
24	Binary search template	Classic template plus search-on-answer.	Google: justify pattern choice. Apple: code hygiene. Goldman: avoid overthinking.
25	Binary search on answer	Koko or Ship Packages in D Days. Google loves the reasoning.	Google: justify pattern choice. Apple: code hygiene. Goldman: avoid overthinking.
26	Rotated array + matrix search	Fast pattern recognition day.	Google: justify pattern choice. Apple: code hygiene. Goldman: avoid overthinking.
27	Mock 4	1 interval + 1 binary-search problem. 50-minute timed round.	Google: justify pattern choice. Apple: code hygiene. Goldman: avoid overthinking.
28	Tree traversals	Iterative inorder/preorder/postorder. Apple lens: code clarity.	Google: recursion invariants. Apple: debug like an owner. Goldman:

			use memorized templates.
29	Tree BFS/DFS	Level order, max depth, diameter.	Google: recursion invariants. Apple: debug like an owner. Goldman: use memorized templates.
30	BST + recursion	Validate BST. State bounds correctly.	Google: recursion invariants. Apple: debug like an owner. Goldman: use memorized templates.
31	Trie	Implement Trie. Tie to etcd/Consul prefix matching.	Google: recursion invariants. Apple: debug like an owner. Goldman: use memorized templates.
32	Cache design	LRU Cache. This is high-value for all three, especially DevOps/SRE.	Google: recursion invariants. Apple: debug like an owner. Goldman: use memorized templates.
33	Design combo	Combine LRU + rate limiter in one 45-minute session.	Google: recursion invariants. Apple: debug like an owner. Goldman: use memorized templates.
34	Debugging round	Take 3 past solutions and intentionally introduce bugs; fix them quickly.	Google: recursion invariants. Apple: debug like an owner. Goldman: use memorized templates.
35	Google-focused mock	1 graph/DP-ish medium + 1 binary-search/trees medium. Narration and testing graded hard.	Each day already company-specific. Treat these as calibration rounds, not study sessions.
36	Apple-focused mock	1 coding problem + 15-minute debugging/edge-case follow-up + 10-minute systems bridge.	Each day already company-specific. Treat these as calibration rounds, not study sessions.
37	Goldman-focused mock	HackerRank-style 2 to 3 shorter problems under tighter time pressure.	Each day already company-specific. Treat these as calibration rounds, not study sessions.

38	Behavioral + project bridge	Prepare 5 stories: outage, automation, debugging, scaling, disagreement. Attach one coding pattern to each.	Each day already company-specific. Treat these as calibration rounds, not study sessions.
39	Weakness sweep	Re-solve your 5 ugliest problems from memory. No hints.	Each day already company-specific. Treat these as calibration rounds, not study sessions.
40	Final simulation day	Do the company-specific round for the firm you are applying to next week. Then stop cramming and review notes.	Each day already company-specific. Treat these as calibration rounds, not study sessions.

Problem bank by company priority

Problem family	Google	Apple	Goldman Sachs
Graphs / BFS / DFS	Very high	High	Medium-high
Heaps / priority queues	High	High	High
Sliding window	High	Medium-high	Very high
Intervals / two pointers	High	Medium	Very high
Trees / recursion / BST	Very high	Medium-high	Medium
Binary search	Very high	Medium-high	High
Trie / prefix structures	Medium	Medium-high	Low-medium
Cache / rate limiter design	High	Very high	High
Debugging follow-up	Medium	Very high	Medium
System bridge discussion	High	High	Medium

Problem count target: around 32 to 40 total solves, but repeat at least 10 of them until you can solve them cleanly without prompts. Repetition beats volume here.

Mock interview format

Google mock

- 5 minutes clarify + plan
- 30 minutes solve medium-to-medium-hard problem with narration
- 10 minutes edge-case and optimization follow-up
- 10 minutes second shorter problem or extension
- 5 minutes recap time/space and tradeoffs

Apple mock

- 30 minutes coding
- 15 minutes debugging or refactor request
- 10 minutes reliability/system follow-up such as cache invalidation, retries, metrics, or rate limiting
- 5 minutes wrap-up

Goldman mock

- 45 to 60 minute OA simulation
- 2 or 3 shorter questions
- little to no hand-holding
- strict timer
- afterward, rewrite one solution for clarity

Behavioral and resume bridge

- Google: keep stories crisp and structured. Show ambiguous problem solving, strong collaboration, and principled tradeoffs.
- Apple: emphasize ownership, debugging discipline, reliability improvements, and quality standards.
- Goldman Sachs: emphasize execution, production support, performance under pressure, and measurable operational impact.
- For all three, prepare one clean story each for incident response, automation, scale, conflict, and a time you found a hidden failure mode before customers did.

Resume bridge line examples: “Reduced MTTR by X% through alert routing + runbook automation,” “Handled Y TPS with <Z ms p99,” “Cut deployment rollback time by X minutes.”

Offer-week checklist

- Have 8 to 10 problems you can solve perfectly from memory.
- Have one spoken template for graph, heap, sliding window, interval, binary search, tree, and LRU/rate-limiter design.
- Have a one-page cheat sheet with Python snippets: deque BFS, heap patterns, binary search, interval merge, trie skeleton, OrderedDict LRU.

- Know exactly which company version of the mock you are running the night before.
- Stop learning new topics in the final 48 hours. Rehearse, do not expand.

Tailoring basis: your uploaded roadmap emphasized 7 core DSA patterns, complexity narration, SRE framing, and a 40-problem bank; this version preserves that structure but reweights it for Google, Apple, and Goldman Sachs.

Notes behind the tailoring

Your uploaded roadmap emphasized a narrow DSA slice for SRE/DevOps interviews: graphs, heaps, sliding windows, two pointers/intervals, trees/tries, light DP, and binary search, plus interview narration and SRE framing.

Google Careers provides a general interview-prep resources page rather than a narrow role-specific shortcut list, which supports a fundamentals-first prep approach.

Apple current software-engineering listings mention a common interview process focused on core fundamentals and programming ability, and Apple has active DevOps/SRE and traffic/network proxying roles that make practical systems-adjacent preparation relevant.

Goldman Sachs says Engineering candidates may complete a HackerRank-based assessment and, if successful, continue to video and final interviews, which supports timer-heavy OA practice.